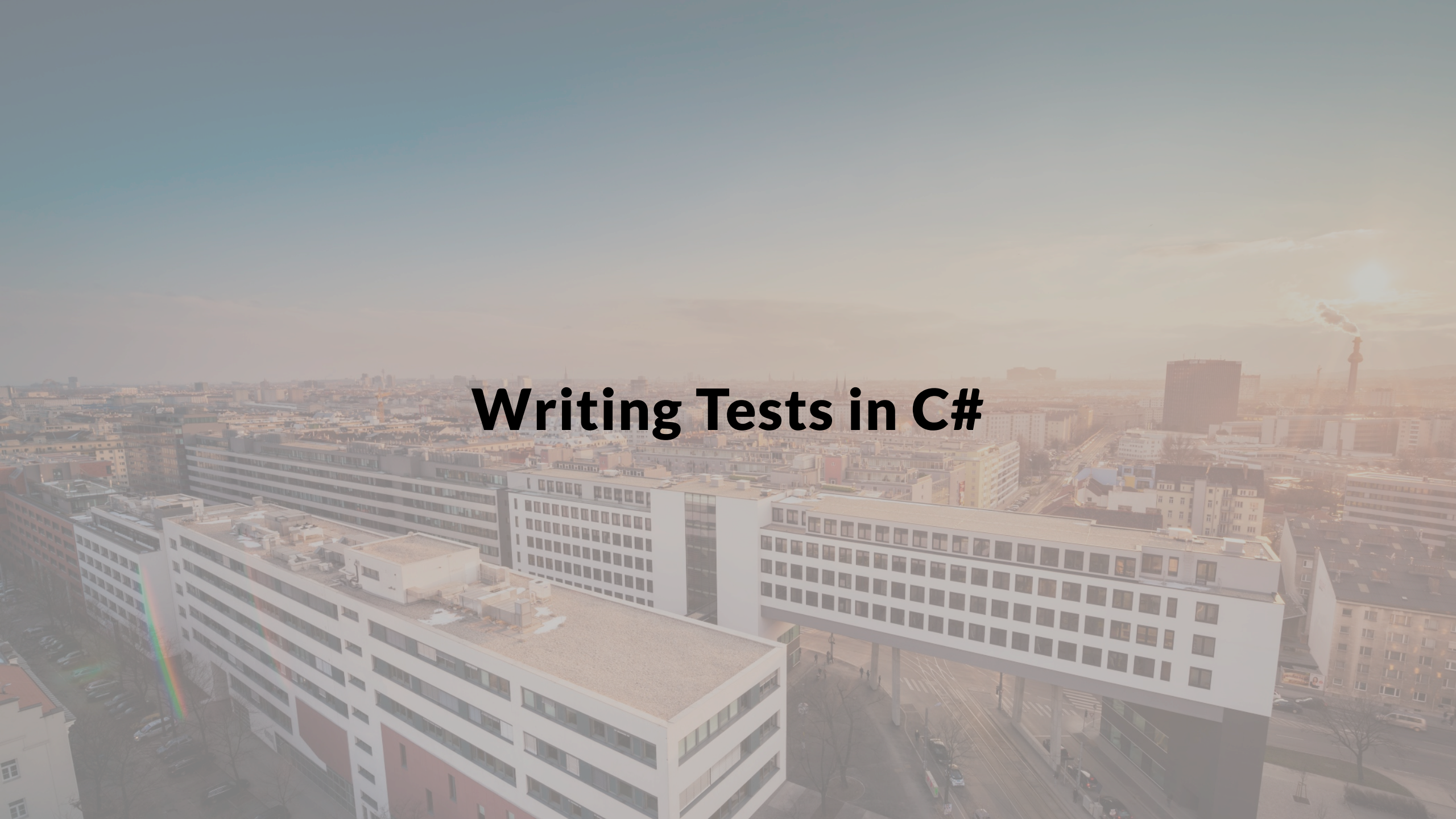


Testing C#

SWEN1 | BIF3

Table of Contents

- Writing Tests in C#
- Hands-On Testing in C#



Writing Tests in C#

Software Engineering - Unit Testing

Unit testing has a great effect on the quality of your code when it's an integral part of your software development workflow.

As soon as you write a function or other block of application code, create unit tests that verify the behavior of the code in response to standard, boundary, and incorrect cases of input data, and that check any explicit or implicit assumptions made by the code.

Software Engineering - Unit Testing - C#

With test driven development (TDD), you create the unit tests before you write the code, so you use the unit tests as both design documentation and functional specifications.

Visual Studio installs the Microsoft unit testing frameworks for managed and native code. Use a unit testing framework to create unit tests, run them, and report the results of these tests. Rerun unit tests when you make changes to test that your code is still working correctly.

For this exercise we'll use NUnit testing framework.

Writing Unit Tests

Naming conventions:

original project: MyProject

↓

test project: MyProject.Test

original class: MyClass

↓

test class: TestMyClass

original method: MyMethod

↓

test method: testMyMethod_scenarioWithResult

Developers perspective:

- easy to write
- reliable
- readable
- fast
- repeatable
- truly unit tests (not integration, no dependencies)

Programming Guides

- Verification of the software
- tests small “units” independently (see: mocking)
- should run automated
- can check state and result values
- requires/enforces good software design (interfaces)

Code-Coverage (amount of code covered by tests):

- 100% … not always possible
- 90% … is often fine

Refactoring:

is structural improvement while not changing the functionality to improve:

- readability
- understandability
- maintenance
- extendibility
- error handling

Refactoring benefits strongly from automated unit testing.

Assert Statements

- `Assert.Fail([message])`: Let the method fail. Might be used to check that a certain part of the code is not reached or to have a failing test before the test code is implemented. The message parameter is optional.
- `Assert.That(condition)`: Checks that the boolean condition is true.
- `Assert.That(!condition)`: Checks that the boolean condition is false.
- `Assert.That(actual, Is.EqualTo(expected))`: Tests that two values are the same. Note: for arrays the
- `Assert.That(actual, Is.GreaterThan(expected))`: Tests that the actual value is greater than the expected.
- `Assert.That(actual, Is.GreaterThan(expected))`: Tests that the actual value is les than the expected.
- `ClassicAssert.IsNull(reference)`: Checks that the object is null.
- `ClassicAssert.IsNotNull(reference)`: Checks that the object is not null.

Testing in C# with NUnit

- NUnit is a unit-testing framework for all .Net languages.
- Initially ported from JUnit, the current version, has already been completely rewritten.
- The NUnit Project is a member of the .NET Foundation.
- The various NUnit packages passed 100 million downloads on NuGet.org.
- NUnit is Open Source software and NUnit 3 is released under the MIT license.

https://docs.nunit.org/articles/nunit/intro.html

Samples

```
namespace NUnit.Framework.Tests
{
  [TestFixture]
  public class AssertSyntaxTests
  {
    [Test]
    public void IsNull()
    {
      object nada = null;
      // Classic syntax
      ClassicAssert.IsNull(nada);

      // Constraint Syntax
      Assert.That(nada, Is.Null);
    }
  }
}
```



Hands-On Testing in C#

Software Engineering - Unit Testing - C#

Demo Project

- Create fighters with a lightsaber and a specific saber color
- Let the fighters fight with specific aims
- Combat and game mechanics to decide who won the fight



Software Engineering - Unit Testing - C#

To add a unit test project to a solution:

- In Solution Explorer, right-click on the solution and choose Add > New Project.
- Use the project template search box to find a unit test project template for the test framework that you want to use.
- On the next page, name the project. To test the LightSaberFight project of our example, you could name the project LightSaberFightTests.
- In your unit test project, add a reference to the code project under test, in our example to the LightSaberFight project.
- To create the reference to the code project:
  - Select the project in Solution Explorer.
  - On the Project menu, choose Add Reference.
  - On the Reference Manager dialog box, open the Solution node and choose Projects. Select the code project name and close the dialog box.

Software Engineering - Unit Testing - C#

The unit test framework that you use and Visual Studio IntelliSense will guide you through writing the code for your unit tests for a code project.

To run in Test Explorer, most frameworks require that you add specific attributes to identify unit test methods. The frameworks also provide a way—usually through assert statements or method attributes—to indicate whether the test method has passed or failed.

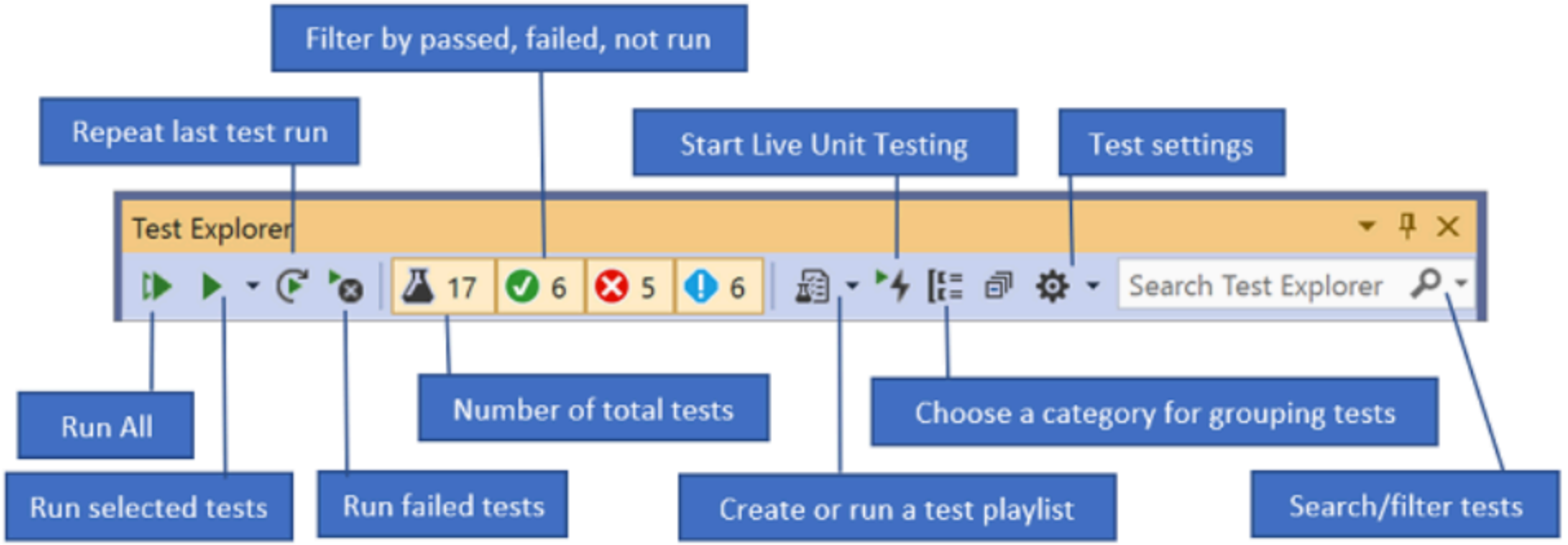
Other attributes identify optional setup methods that are at class initialization and before each test method and teardown methods that are run after each test method and before the class is destroyed.

The AAA (Arrange, Act, Assert) pattern is a common way of writing unit tests for a method under test.

- The Arrange section of a unit test method initializes objects and sets the value of the data that is passed to the method under test.
- The Act section invokes the method under test with the arranged parameters.
- The Assert section verifies that the action of the method under test behaves as expected.

Software Engineering - Unit Testing - C#

The Test Explorer toolbar helps you discover, organize, and run the tests that you are interested in.



Software Engineering - Unit Testing - C#

To determine what proportion of your project's code is actually being tested by coded tests such as unit tests, you can use the code coverage feature. To guard effectively against bugs, your tests should exercise or 'cover' a large proportion of your code.

The code coverage feature is available only in Visual Studio Enterprise edition.

Possible free solution to incorporate code coverage into your project - FineCodeCoverage:
https://marketplace.visualstudio.com/items?itemName=FortuneNgwenya.FineCodeCoverage